Name _____

CSCI 332, Fall 2026
Exam 1

This exam consists of the following sections:

1. Stable Matching (25 points)
2. Algorithm Analysis (25 points)
3. Recursion (25 points)
4. Backtracking and 1D Dynamic Programming (25 points)

You may use a double sided 3x5 handwritten notecard of notes during the test but no other resources.

Good luck!

Section 1 (Stable Matching)

Questions 1 and 2 refer to the Gale-Shapley algorithm for stable matching.

```
Gale-Shapley(preference lists for n hospitals and n students)
  Let M be an empty matching
  While some hospital h is unmatched and hasn't proposed to every student:
      Let s be the first student on h's list to whom h has not yet proposed
      If s is unmatched:
          Add (h,s) to M
      Else s is matched to h':
          If s prefers h to h':
              Remove (h',s) from M
              Add (h,s) to M
          Else s prefers h' to h:
              Do nothing
  Return M
```

1. (10 points) What is the matching returned by Gale-Shapley for the following input?

Hospital	**1st**	**2nd**	**3rd**
h_1	s_1	s_2	s_3
h_2	s_1	s_2	s_3
h_3	s_2	s_3	s_1
Student	**1st**	**2nd**	**3rd**
s_1	h_3	h_2	h_1
s_2	h_1	h_2	h_3
s_3	h_1	h_2	h_3

Choose the matching returned:

(a) $\{(s_1,h_2),(s_2,h_1),(s_3,h_3)\}$

(b) $\{(s_1,h_3),(s_2,h_1),(s_3,h_2)\}$

(c) $\{(s_1,h_1),(s_2,h_2),(s_3,h_3)\}$

(d) $\{(s_1,h_2),(s_2,h_3),(s_3,h_1)\}$

2. (5 points) Does a distinct stable matching exist for this input? Circle: Yes or No. The input is copied again in case that is useful.

Hospital	**1st choice**	**2nd choice**	**3rd choice**
h_1	s_1	s_2	s_3
h_2	s_1	s_2	s_3
h_3	s_2	s_3	s_1
Student	**1st choice**	**2nd choice**	**3rd choice**
s_1	h_3	h_2	h_1
s_2	h_1	h_2	h_3
s_3	h_1	h_2	h_3

3. (5 points) Is it true that in every input to the stable matching problem, if a hospital h ranks a student s first and that same student s ranks that same h first, then (h,s) is matched in every possible stable matching of this input? Circle True or False.

4. (5 points) Is it true that every stable matching contains at least one pair (h,s) so that h ranks s first and s ranks h first? Circle True or False.

Section 2 (Algorithm Analysis)

5. (5 points) Order the following functions from asymptotically smallest to asymptotically largest by labeling the smallest with a 1 and increasing from there. **If two functions are asymptotically equal (one is Θ of the other), give them the same number**. Two are filled as an example for you.

   ___ $\log_4 n^3$
   ___ $n + \log n$
   _1_ $100^{100!}$
   _1_ 5
   ___ $\log_{10} 100^{10}$
   ___ $\log_2 n$

6. (5 points) Select all functions $f(n)$ such that the runtime of Algorithm-1 below is $\Theta(f(n))$.

```
Algorithm-1(array A of length n):
    Result = 0
    For each i in 1 to n:
        For each j in 1 to i:
            Set Result to A[j] times Result
    Return Result
```

   A. n
   B. $\log n$
   C. n^2
   D. n^3
   E. $n!$

7. (5 points) Recall that the definition of big O is as follows: *$f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$.* Select all pairs n_0 and c can be used to show that $4n + 10$ is $O(n)$.

   A. $n_0 = 1$, $c = 4$
   B. $n_0 = 10, c = 4$
   C. $n_0 = 10$, $c = 5$
   D. $n_0 = 1$, $c = 1000$
   E. $n_0 = 100$, $c = 1$

8. (5 points) Suppose you know that an algorithm has a best-case runtime that is $O(n^2)$. For each of the following, decide whether it is definitely true, definitely false, or could be true or false and circle your choice.

   - The algorithm's worst-case runtime is $\Omega(n)$. T, F, T or F
   - The algorithm's worst-case runtime is $\Omega(n^3)$. T, F, T or F
   - The algorithm's best-case runtime is $O(n^3)$. T, F, T or F

9. (5 points) You suspect that in the most recent update to your favorite software package, the implementation of your favorite function `do-stuff` was replaced by a $\Theta(n^2)$ algorithm, whereas it was a $\Theta(n^3)$ algorithm before. Whatever the runtime is, you know that it is the same for all input classes (that is, the worst case and best case runtimes are equal). Unfortunately, the package is closed-source and you cannot inspect the source code; you can only run the function on various-sized inputs and record its runtime on those inputs.

   Describe an experiment that you could run to verify your hunch. That is, tell us what kinds of input sizes you will run `do-stuff` on and how you will use their runtimes to determine whether the algorithm is $\Theta(n^2)$ or $\Theta(n^3)$.

Section 3 (Recursion)

10. (15 points)

Consider the recurrence $T(n) = 2T(n/3) + n^3, T(1) = 1$.

(a) (5 points) In the left boxes, draw the first three levels of this recursion tree. In the boxes next to them, write the total work on that level.

Level 0	Sum level 0
Level 1	Sum level 1
Level 2	Sum level 2

(b) (3 points) What is the work per level in terms of level ℓ?

(c) (2 points) Is the work per level (circle one)

- increasing (work is dominated by leaves)
- decreasing (work is dominated by root)
- the same (work is the same at every level)

(d) (3 points) What is the total number of levels in the tree? (fill in the base and the argument of the logarithm)

number of levels = log_____(_____)

(e) (2 points) What is the closed-form (non-recursive form) of $T(n)$?

Recall the algorithm for binary search:

```
BinarySearch(sorted A[1..n], t):
  ℓ = 1, r = n
  while ℓ ≤ r:
      m = ⌊(ℓ+r)/2⌋
      if A[m] > t: ⟨⟨go left⟩⟩
          r = m − 1
      else if A[m] < t: ⟨⟨go right⟩⟩
          ℓ = m + 1
      else: ⟨⟨equal⟩⟩
          return m
  return −1
```

In this problem, you will instead solve the following problem: Suppose you are given a sorted array of n distinct numbers that has been rotated k steps, for some unknown integer k between 1 and $n-1$. That is, you are given an array $A[1..n]$ such that some prefix $A[1..k]$ is sorted in increasing order, the corresponding suffix $A[k+1..n]$ is sorted in increasing order, and $A[n] < A[1]$. For example, you might be given the following 16-element array (where $k = 10$):

9	13	16	18	19	23	28	31	37	42	1	3	4	5	7	8

Your task is to determine what k is in $O(\log n)$ time.

11. (10 points) You will still look at the midpoint m between your current ℓ and r indices in the array. How will you decide whether to go left or right?

Section 4 (Backtracking and 1D Dynamic Programming)

In your country, there are only coins of denominations 1, 3, and 4. You would like to write an algorithm to solve the following problem: Given a positive integer n, what is the fewest number of coins you can use to represent n?

To solve this problem, you choose the following English definition of the recursive subproblems.

Let $MinCoins(i)$ be the minimum number of coins needed to represent the amount i.

12. (2 points) What is $MinCoins(6)$?

13. (8 points) Write the recursive definition for $MinCoins(i)$. You may not need every line provided.

$$MinCoins(i) = \begin{cases} _____ & if\ i_____ \\ _____ & if\ i_____ \\ _____ & if\ i_____ \\ _____ & if\ i_____ \\ _____ & if\ i_____ \end{cases}$$

Given the following (meaningless) recursive definition of a subproblem

$$doh(i) = \begin{cases} 4 & if\ i = n \\ \text{H[n-1]} & if\ i = n-1 \\ 0 & if\ i = n-2 \\ \min(2 \cdot H[i] + doh(i+1), doh(i+3)) & if\ i < n-2 \end{cases}$$

14. (2 points) We can store solutions to the subproblems in an array. What direction should we fill it (left to right or right to left)?

15. (8 points)

Using the above information, finish the pseudocode to compute $doh(1)$ for a given $H[1..n]$.

```
MemoizedDoh(H[1..n]):
  Let array M be an array of length n
  # do we have any base cases?




  # how should we fill in the rest of the array?




  # what should we return to get the final answer?
```

The subset-sum problem is as follows: Given a set X of positive integers and a target integer T, is there a subset of elements in X that add up to T? For example, if $X = \{8, 6, 7, 5, 3, 10, 9\}$ and $T = 15$, the answer is True, because the subsets $\{8, 7\}$ and $\{6, 9\}$ and $\{7, 5, 3\}$ all sum to 15.

16. (1 point) Find one more subset that sums to 15 in X.

17. (4 points) Consider the following subproblem: **Let $SubsetSum(S, t)$ be True if the set S contains a subset that sums to t and False if not.** To write the recursive case of a backtracking algorithm to solve subset-sum, we observe that if we have a set S and a target t, we can remove any single element x from S. Then, our recursive algorithm should explore two choices, **which you should state as new calls to SubsetSum()**. Make sure you specify the two parameters that SubsetSum takes in.

    - Choice 1:

    - Choice 2: